

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: df = pd.read_csv("HR_comma_sep.csv")
```

```
In [5]: df
```

```
Out[5]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_s
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	
...	...	...	...	...	...
14994	0.40	0.57	2	151	
14995	0.37	0.48	2	160	
14996	0.37	0.53	2	143	
14997	0.11	0.96	6	280	
14998	0.37	0.52	2	158	

14999 rows × 10 columns



```
In [4]: df.head()
```

```
Out[4]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	



```
In [5]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation         14999 non-null  float64
2   number_project          14999 non-null  int64
3   average_monthly_hours  14999 non-null  int64
4   time_spend_company     14999 non-null  int64
5   Work_accident          14999 non-null  int64
6   left                   14999 non-null  int64
7   promotion_last_5years  14999 non-null  int64
8   sales                   14999 non-null  object
9   salary                  14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB

```

In [6]: `df.describe()`

Out[6]:

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend_company
count	14999.000000	14999.000000	14999.000000	14999.000000	14999.000000
mean	0.612834	0.716102	3.803054	201.050337	201.050337
std	0.248631	0.171169	1.232592	49.943099	49.943099
min	0.090000	0.360000	2.000000	96.000000	96.000000
25%	0.440000	0.560000	3.000000	156.000000	156.000000
50%	0.640000	0.720000	4.000000	200.000000	200.000000
75%	0.820000	0.870000	5.000000	245.000000	245.000000
max	1.000000	1.000000	7.000000	310.000000	310.000000

In [7]: `df.isna().sum()`

Out[7]:

```

satisfaction_level      0
last_evaluation          0
number_project           0
average_monthly_hours    0
time_spend_company      0
Work_accident           0
left                    0
promotion_last_5years    0
sales                    0
salary                   0
dtype: int64

```

In [9]: `df["left"].unique()`

Out[9]: `array([1, 0])`

In [10]: `df["number_project"].unique()`

Out[10]: `array([2, 5, 7, 6, 4, 3])`

```
In [11]: df.satisfaction_level.unique()
```

```
Out[11]: array([0.38, 0.8 , 0.11, 0.72, 0.37, 0.41, 0.1 , 0.92, 0.89, 0.42, 0.45,
                0.84, 0.36, 0.78, 0.76, 0.09, 0.46, 0.4 , 0.82, 0.87, 0.57, 0.43,
                0.13, 0.44, 0.39, 0.85, 0.81, 0.9 , 0.74, 0.79, 0.17, 0.24, 0.91,
                0.71, 0.86, 0.14, 0.75, 0.7 , 0.31, 0.73, 0.83, 0.32, 0.54, 0.27,
                0.77, 0.88, 0.48, 0.19, 0.6 , 0.12, 0.61, 0.33, 0.56, 0.47, 0.28,
                0.55, 0.53, 0.59, 0.66, 0.25, 0.34, 0.58, 0.51, 0.35, 0.64, 0.5 ,
                0.23, 0.15, 0.49, 0.3 , 0.63, 0.21, 0.62, 0.29, 0.2 , 0.16, 0.65,
                0.68, 0.67, 0.22, 0.26, 0.99, 0.98, 1. , 0.52, 0.93, 0.97, 0.69,
                0.94, 0.96, 0.18, 0.95])
```

```
In [12]: df.last_evaluation.unique()
```

```
Out[12]: array([0.53, 0.86, 0.88, 0.87, 0.52, 0.5 , 0.77, 0.85, 1. , 0.54, 0.81,
                0.92, 0.55, 0.56, 0.47, 0.99, 0.51, 0.89, 0.83, 0.95, 0.57, 0.49,
                0.46, 0.62, 0.94, 0.48, 0.8 , 0.74, 0.7 , 0.78, 0.91, 0.93, 0.98,
                0.97, 0.79, 0.59, 0.84, 0.45, 0.96, 0.68, 0.82, 0.9 , 0.71, 0.6 ,
                0.65, 0.58, 0.72, 0.67, 0.75, 0.73, 0.63, 0.61, 0.76, 0.66, 0.69,
                0.37, 0.64, 0.39, 0.41, 0.43, 0.44, 0.36, 0.38, 0.4 , 0.42])
```

```
In [13]: df.time_spend_company.unique()
```

```
Out[13]: array([ 3,  6,  4,  5,  2,  8, 10,  7])
```

```
In [14]: df.Work_accident.unique()
```

```
Out[14]: array([0, 1])
```

```
In [15]: df.sales.unique()
```

```
Out[15]: array(['sales', 'accounting', 'hr', 'technical', 'support', 'management',
                'IT', 'product_mng', 'marketing', 'RandD'], dtype=object)
```

```
In [16]: df.salary.unique()
```

```
Out[16]: array(['low', 'medium', 'high'], dtype=object)
```

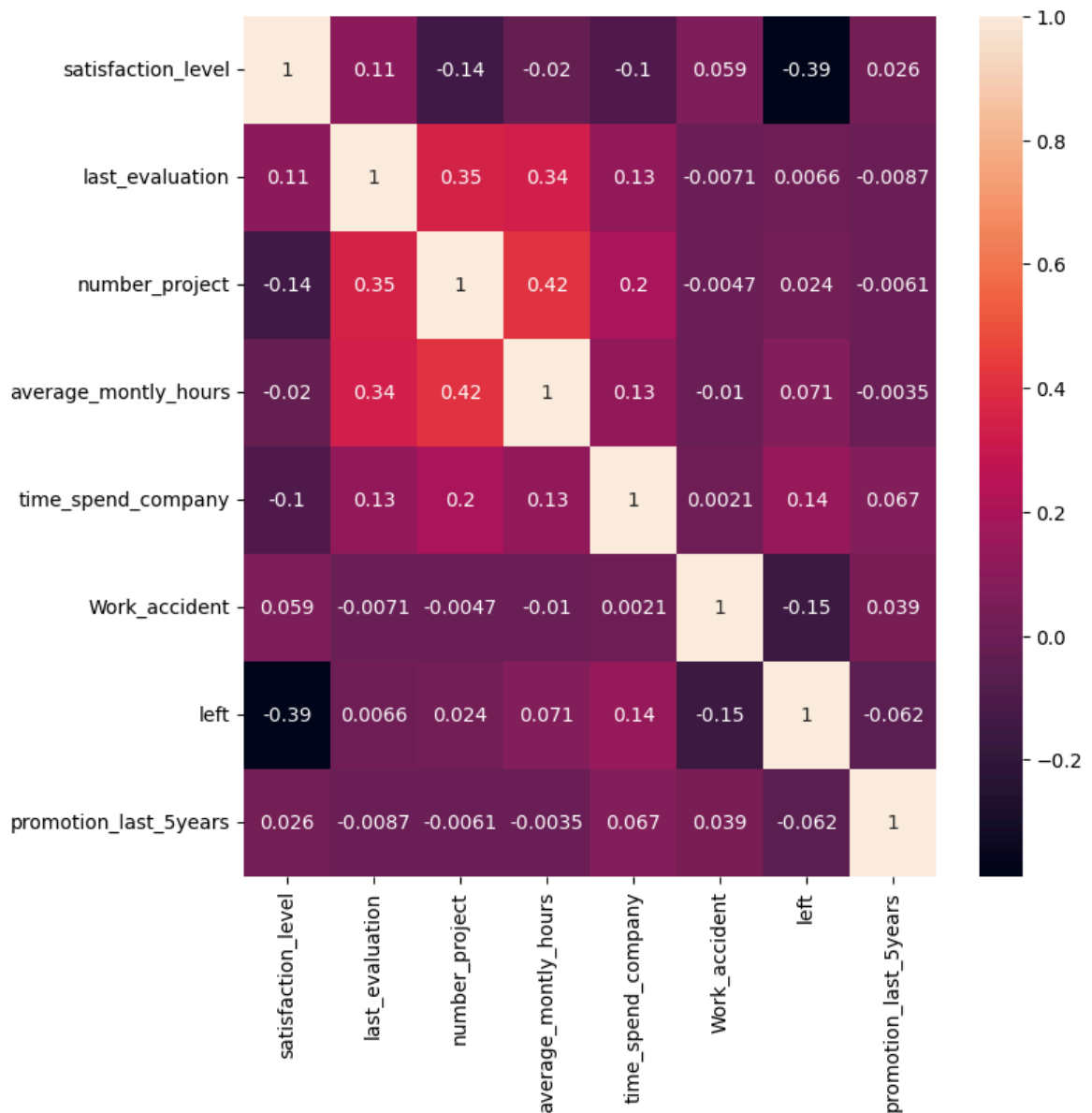
```
In [18]: df.corr(numeric_only=True)
```

```
Out[18]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours
satisfaction_level	1.000000	0.105021	-0.142970	-0.020048
last_evaluation	0.105021	1.000000	0.349333	0.339742
number_project	-0.142970	0.349333	1.000000	0.417211
average_monthly_hours	-0.020048	0.339742	0.417211	1.000000
time_spend_company	-0.100866	0.131591	0.196786	0.006567
Work_accident	0.058697	-0.007104	-0.004741	-0.388375
left	-0.388375	0.006567	0.023787	0.025605
promotion_last_5years	0.025605	-0.008684	-0.006064	0.000000

```
In [20]: plt.figure(figsize=(8,8))
sns.heatmap(df.corr(numeric_only=True), annot=True)
```

Out[20]: <Axes: >



```
In [21]: df1 = df.groupby(["sales"])[["left"]].value_counts().reset_index(name="count")
df1 = pd.DataFrame(df1)
```

```
In [22]: df["sales"].value_counts()
```

```
Out[22]: sales
sales      4140
technical  2720
support    2229
IT         1227
product_mng 902
marketing   858
RandD      787
accounting  767
hr         739
management 630
Name: count, dtype: int64
```

```
In [23]: dft = df["sales"].value_counts().reset_index(name="Total")
```

```
In [24]: dft = dft.rename(columns={"index": "sales"})
```

```
In [25]: dft
```

```
Out[25]:
```

	sales	Total
0	sales	4140
1	technical	2720
2	support	2229
3	IT	1227
4	product_mng	902
5	marketing	858
6	RandD	787
7	accounting	767
8	hr	739
9	management	630

	sales	Total
0	sales	4140
1	technical	2720
2	support	2229
3	IT	1227
4	product_mng	902
5	marketing	858
6	RandD	787
7	accounting	767
8	hr	739
9	management	630

```
In [26]: dfmer = df1.merge(dft,how='left')
```

```
In [27]: dfmer
```

Out[27]:

	sales	left	count	Total
0	IT	0	954	1227
1	IT	1	273	1227
2	RandD	0	666	787
3	RandD	1	121	787
4	accounting	0	563	767
5	accounting	1	204	767
6	hr	0	524	739
7	hr	1	215	739
8	management	0	539	630
9	management	1	91	630
10	marketing	0	655	858
11	marketing	1	203	858
12	product_mng	0	704	902
13	product_mng	1	198	902
14	sales	0	3126	4140
15	sales	1	1014	4140
16	support	0	1674	2229
17	support	1	555	2229
18	technical	0	2023	2720
19	technical	1	697	2720

```
In [28]: dfmer["normal"]=dfmer["count"].div(dfmer["Total"].values)
dfmer["normal"]=dfmer["normal"]*100
```

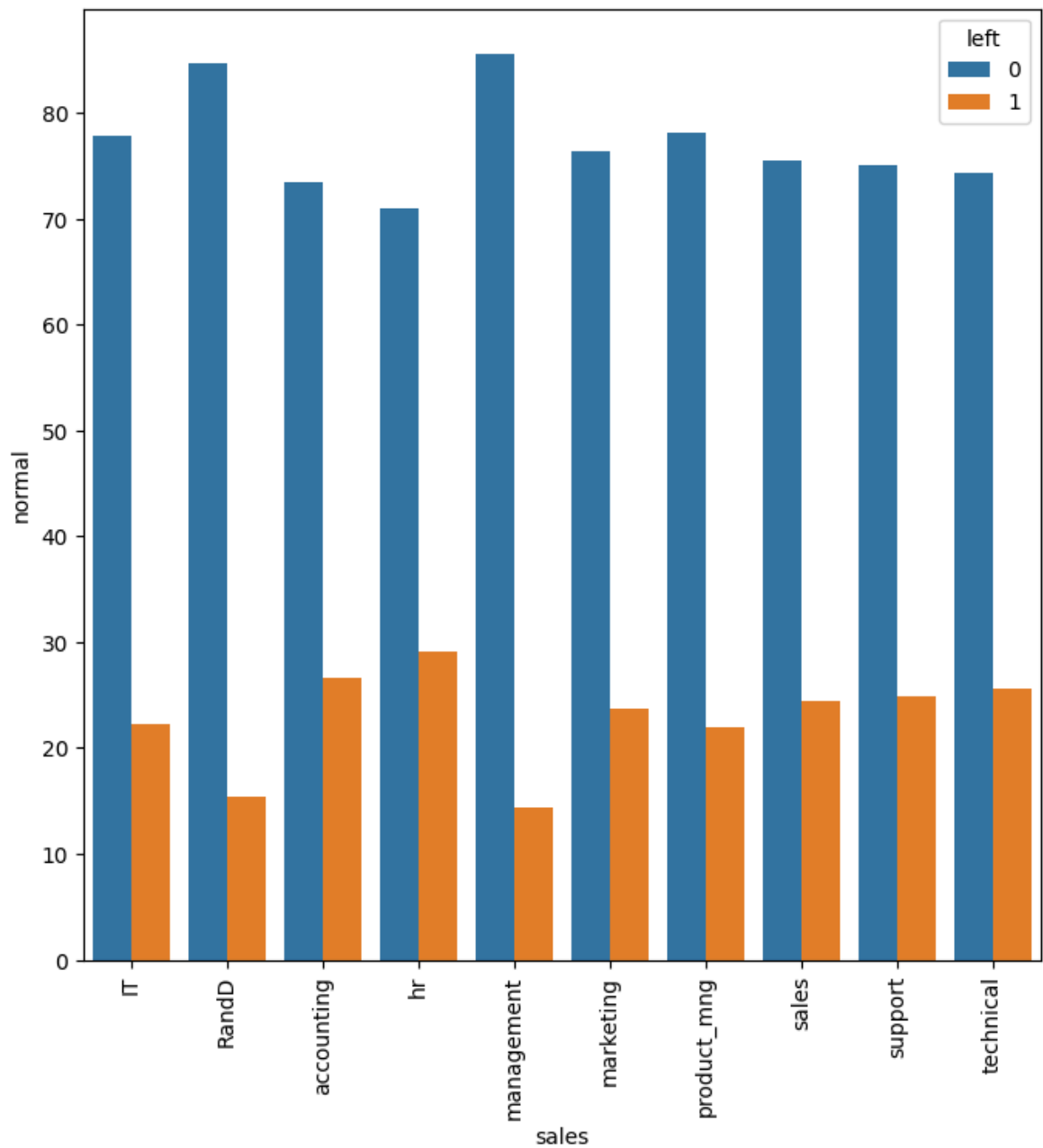
```
In [29]: dfmer
```

Out[29]:

	sales	left	count	Total	normal
0	IT	0	954	1227	77.750611
1	IT	1	273	1227	22.249389
2	RandD	0	666	787	84.625159
3	RandD	1	121	787	15.374841
4	accounting	0	563	767	73.402868
5	accounting	1	204	767	26.597132
6	hr	0	524	739	70.906631
7	hr	1	215	739	29.093369
8	management	0	539	630	85.555556
9	management	1	91	630	14.444444
10	marketing	0	655	858	76.340326
11	marketing	1	203	858	23.659674
12	product_mng	0	704	902	78.048780
13	product_mng	1	198	902	21.951220
14	sales	0	3126	4140	75.507246
15	sales	1	1014	4140	24.492754
16	support	0	1674	2229	75.100942
17	support	1	555	2229	24.899058
18	technical	0	2023	2720	74.375000
19	technical	1	697	2720	25.625000

```
In [30]: plt.figure(figsize=(8,8))
sns.barplot(x='sales',y='normal',hue='left',data=dfmer)
plt.xticks(rotation=90)
```

```
Out[30]: ([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
 [Text(0, 0, 'IT'),
  Text(1, 0, 'RandD'),
  Text(2, 0, 'accounting'),
  Text(3, 0, 'hr'),
  Text(4, 0, 'management'),
  Text(5, 0, 'marketing'),
  Text(6, 0, 'product_mng'),
  Text(7, 0, 'sales'),
  Text(8, 0, 'support'),
  Text(9, 0, 'technical')])
```



```
In [31]: df1.head()
```

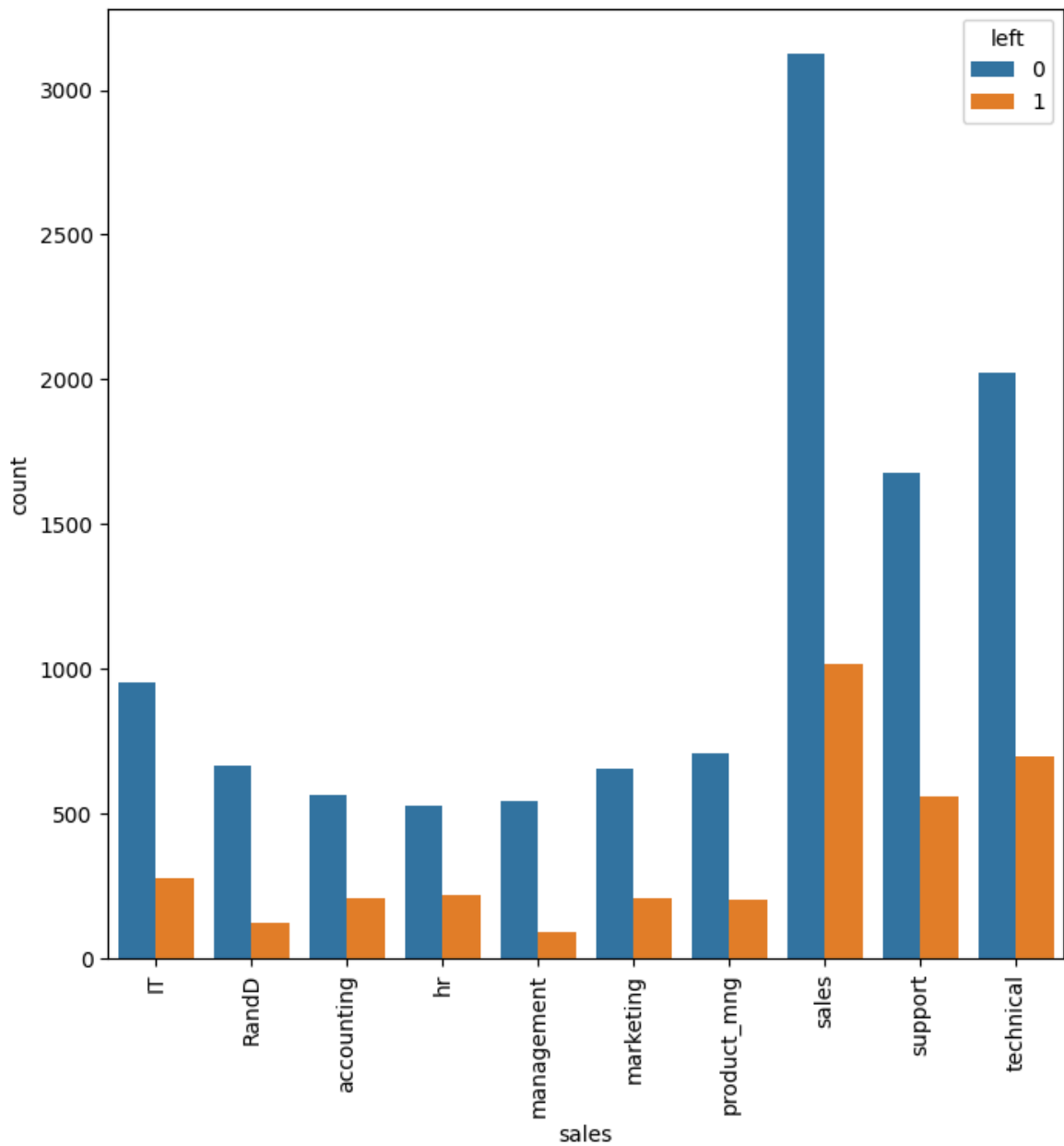
```
Out[31]:
```

	sales	left	count
0	IT	0	954
1	IT	1	273
2	RandD	0	666
3	RandD	1	121
4	accounting	0	563

```
In [32]: plt.figure(figsize=(8,8))
sns.barplot(x='sales',y='count',hue='left',data=df1)
plt.xticks(rotation=90)
```



```
Out[32]: ([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
[Text(0, 0, 'IT'),
Text(1, 0, 'RandD'),
Text(2, 0, 'accounting'),
Text(3, 0, 'hr'),
Text(4, 0, 'management'),
Text(5, 0, 'marketing'),
Text(6, 0, 'product_mng'),
Text(7, 0, 'sales'),
Text(8, 0, 'support'),
Text(9, 0, 'technical')])
```



```
In [33]: df2 = df.groupby(["salary"])[["left"]].value_counts().reset_index(name="count")
df2 = pd.DataFrame(df2)
```

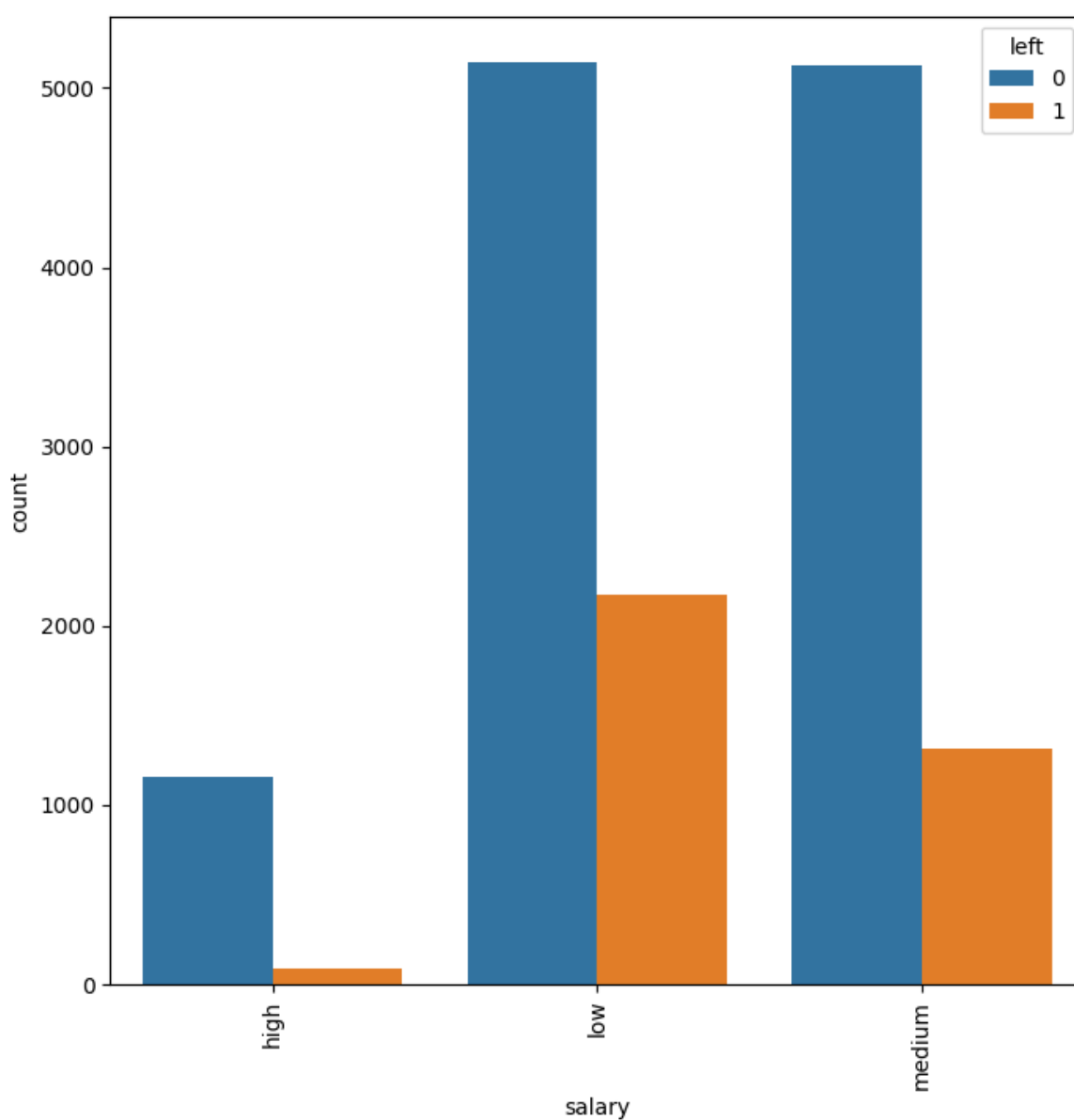
```
In [34]: df2.head()
```

Out[34]:

	salary	left	count
0	high	0	1155
1	high	1	82
2	low	0	5144
3	low	1	2172
4	medium	0	5129

```
In [35]: plt.figure(figsize=(8,8))
sns.barplot(x="salary",y='count',hue="left",data=df2)
plt.xticks(rotation=90)
```

Out[35]: ([0, 1, 2], [Text(0, 0, 'high'), Text(1, 0, 'low'), Text(2, 0, 'medium')])

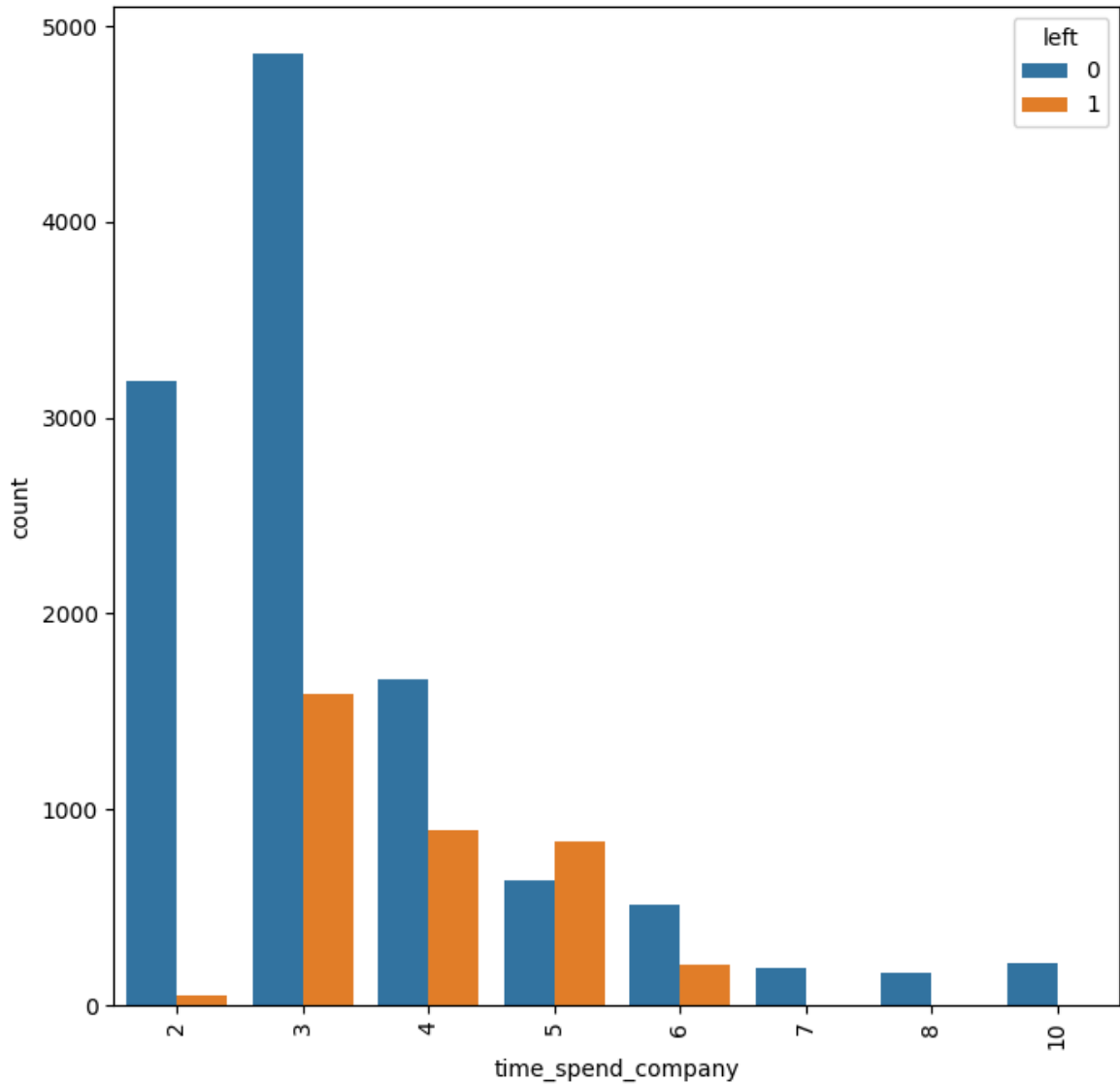


```
In [38]: df3 = df.groupby(["time_spend_company"])[["left"]].value_counts().reset_index(name="count")
df3 = pd.DataFrame(df3)
```

```
In [39]: plt.figure(figsize=(8,8))
sns.barplot(x="time_spend_company",y='count',hue="left",data=df3)
```

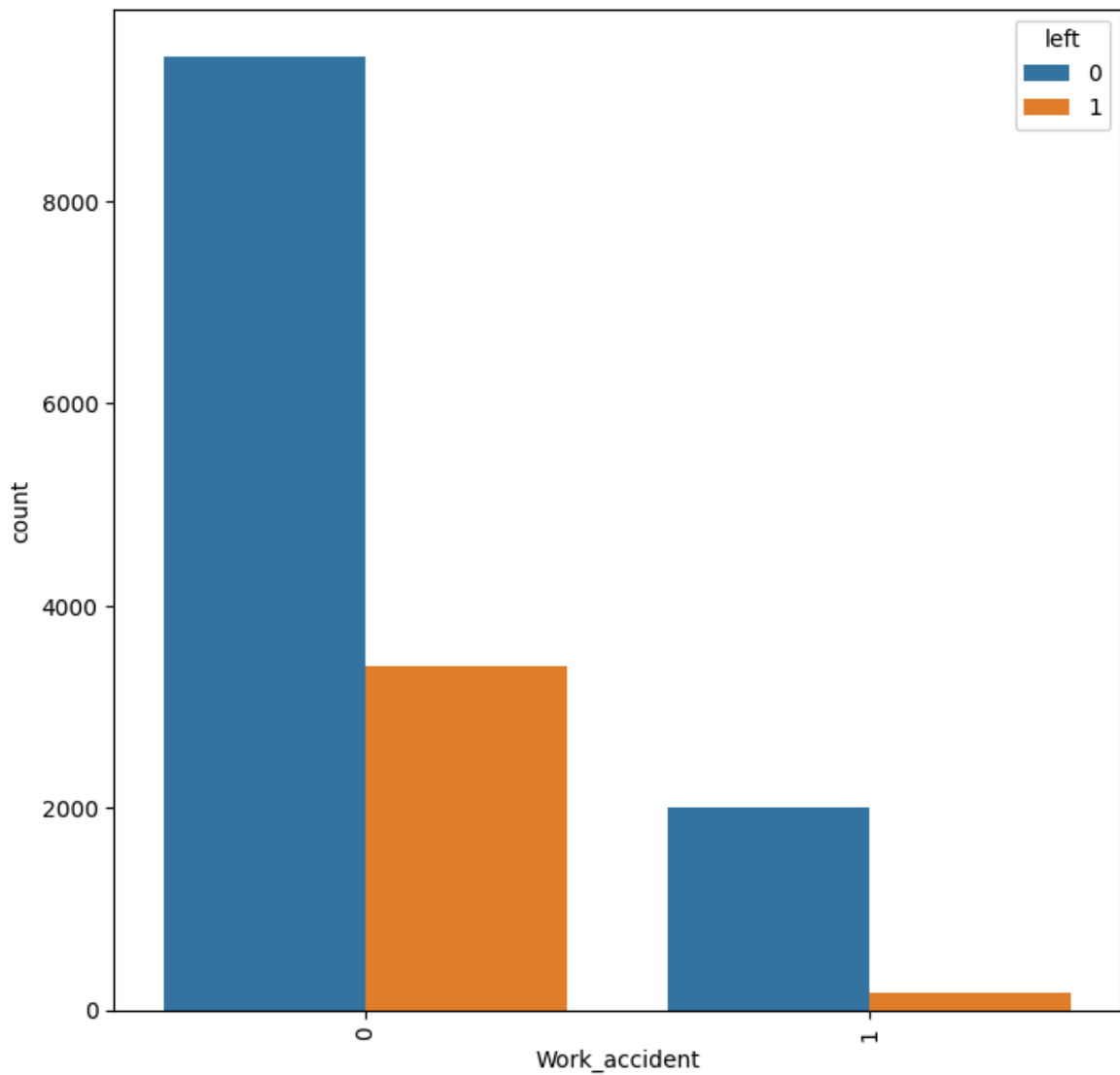
```
plt.xticks(rotation=90)
```

```
Out[39]: ([0, 1, 2, 3, 4, 5, 6, 7],  
          [Text(0, 0, '2'),  
           Text(1, 0, '3'),  
           Text(2, 0, '4'),  
           Text(3, 0, '5'),  
           Text(4, 0, '6'),  
           Text(5, 0, '7'),  
           Text(6, 0, '8'),  
           Text(7, 0, '10')])
```



```
In [40]: plt.figure(figsize=(8, 8))  
sns.countplot(x="Work_accident", hue="left", data=df)  
plt.xticks(rotation=90)
```

```
Out[40]: ([0, 1], [Text(0, 0, '0'), Text(1, 0, '1')])
```

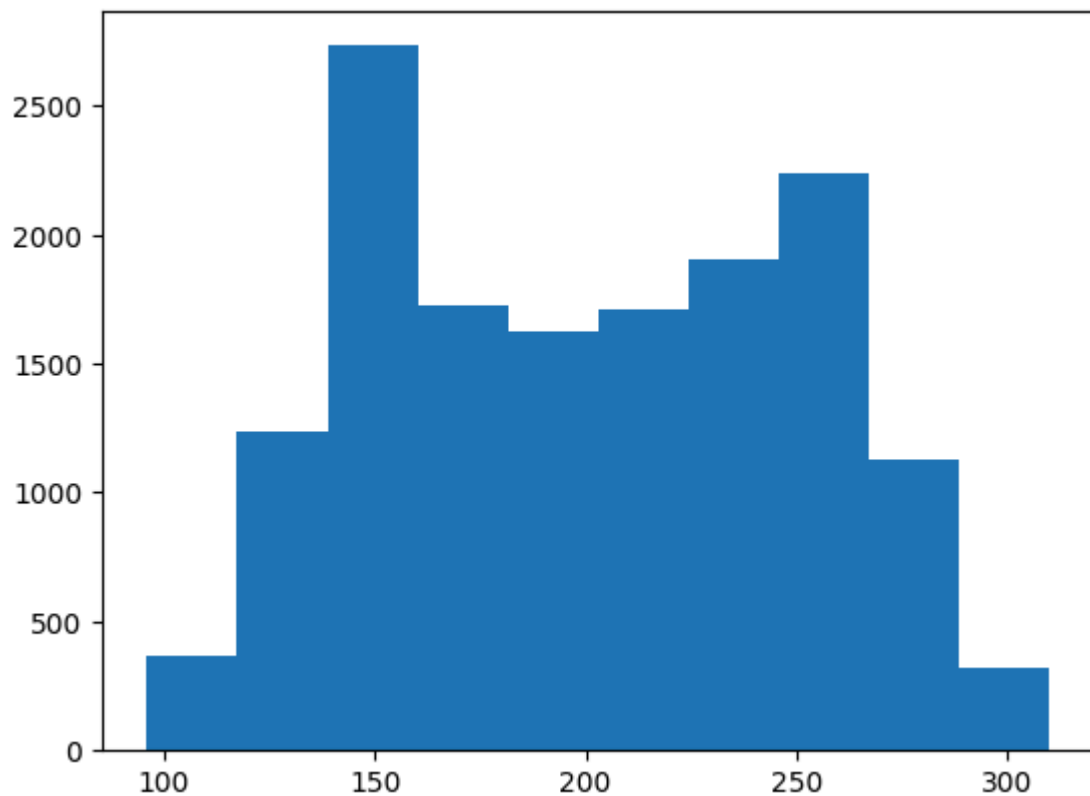


```
In [41]: df.columns
```

```
Out[41]: Index(['satisfaction_level', 'last_evaluation', 'number_project',
               'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left',
               'promotion_last_5years', 'sales', 'salary'],
              dtype='object')
```

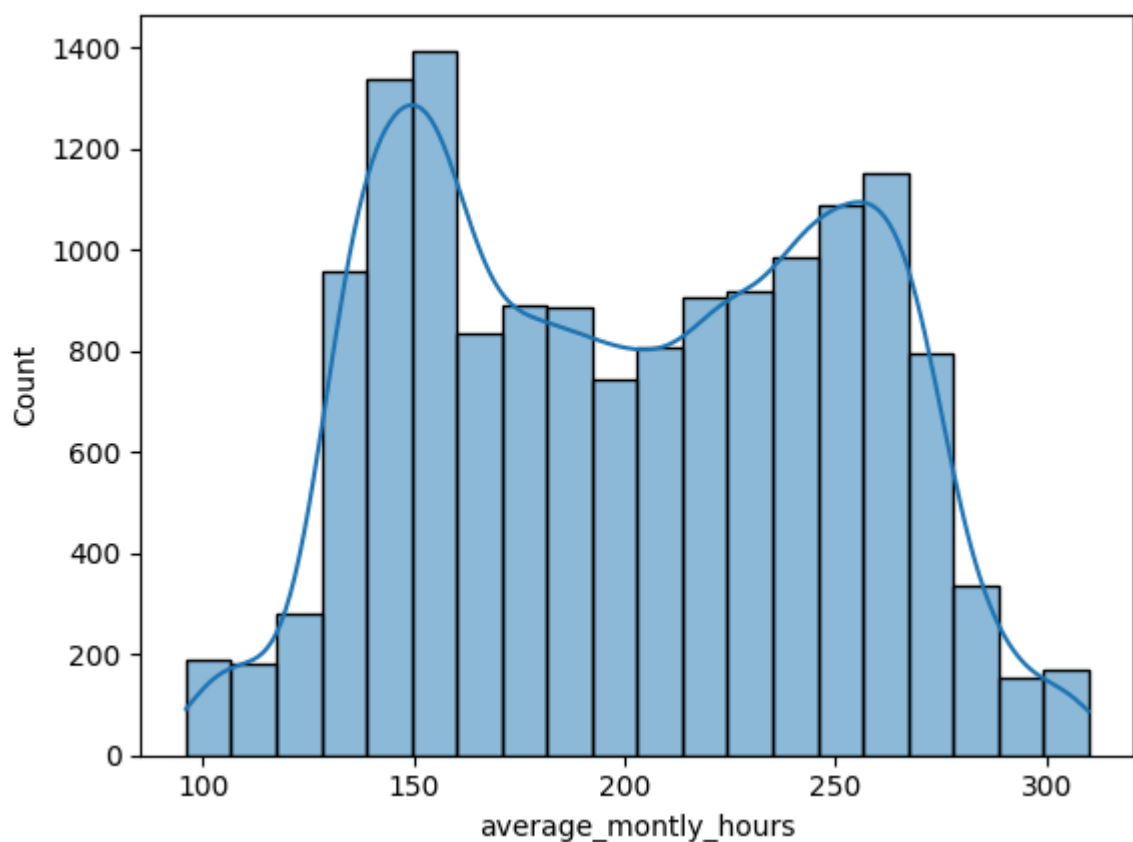
```
In [42]: plt.hist(df["average_monthly_hours"])
```

```
Out[42]: (array([ 367., 1240., 2733., 1722., 1628., 1712., 1906., 2240., 1127.,
               324.]),
          array([ 96. , 117.4, 138.8, 160.2, 181.6, 203. , 224.4, 245.8, 267.2,
               288.6, 310. ]),
          <BarContainer object of 10 artists>)
```



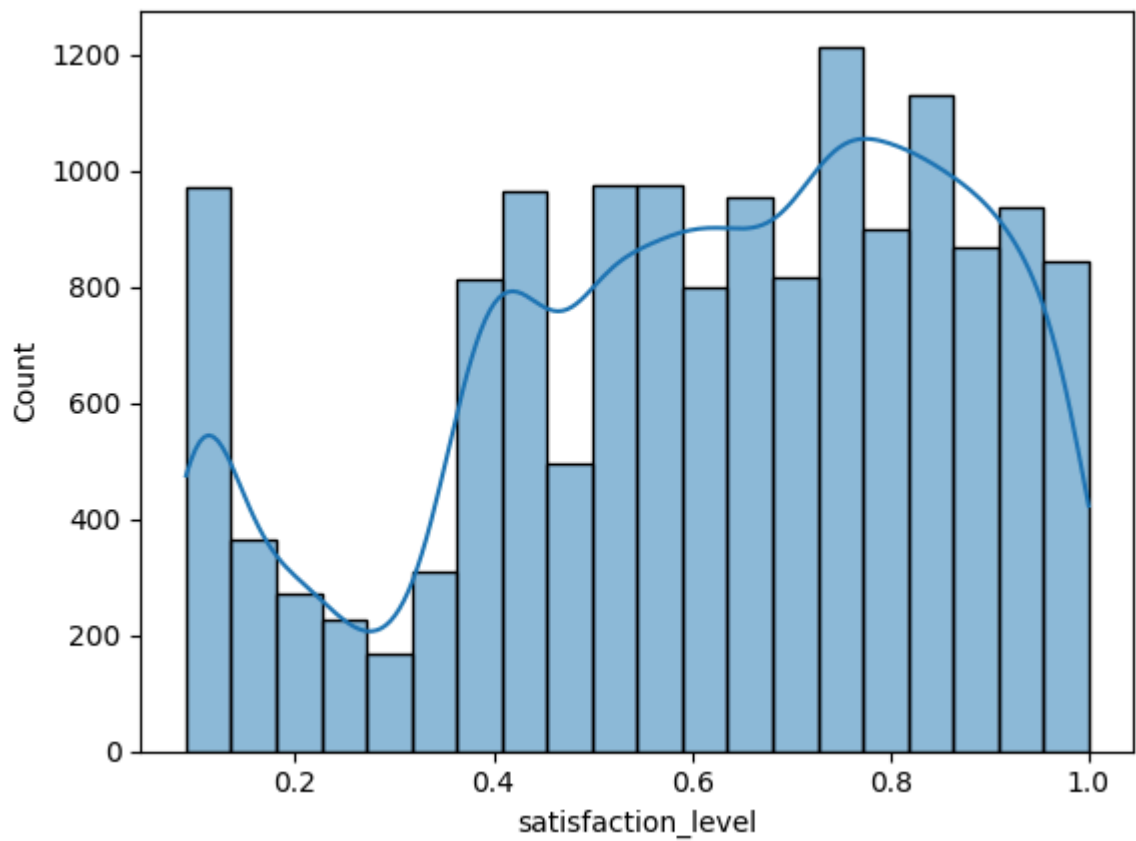
```
In [43]: sns.histplot(data = df,x="average_monthly_hours", kde = True,bins=20)
```

```
Out[43]: <Axes: xlabel='average_monthly_hours', ylabel='Count'>
```



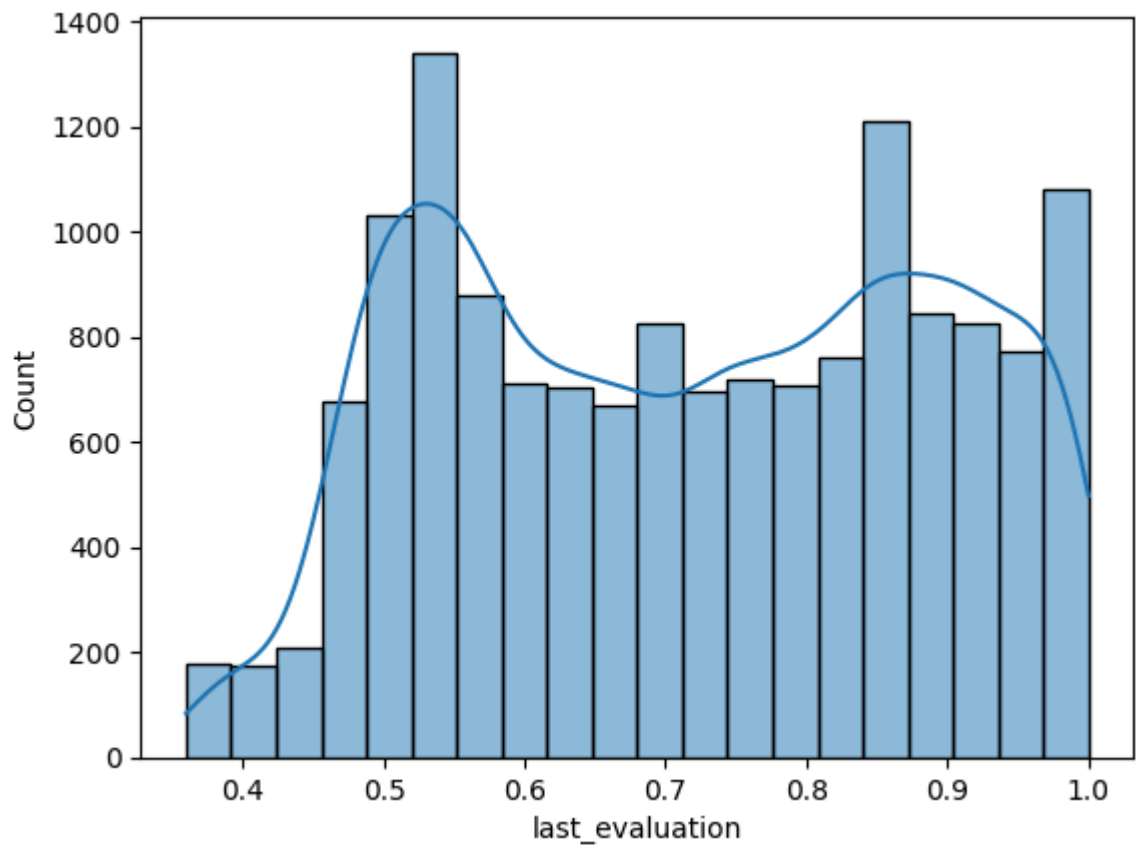
```
In [44]: sns.histplot(data = df,x="satisfaction_level", kde = True,bins=20)
```

```
Out[44]: <Axes: xlabel='satisfaction_level', ylabel='Count'>
```



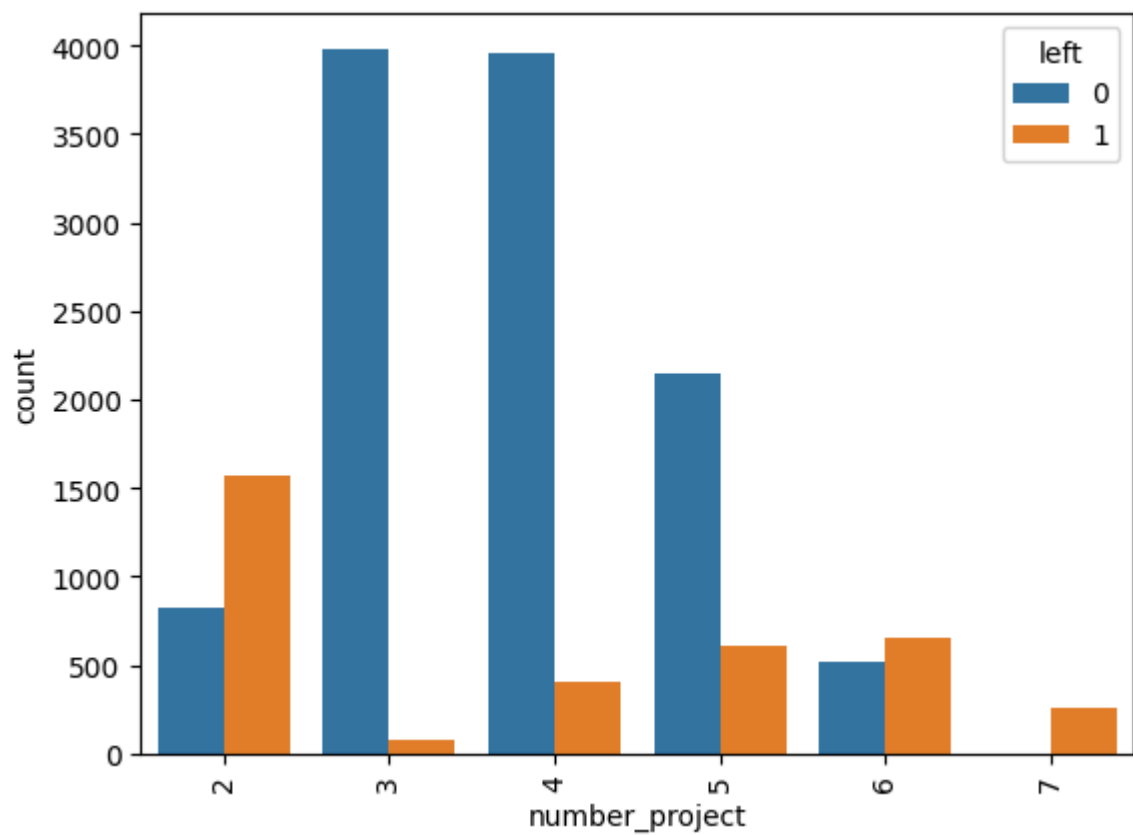
```
In [45]: sns.histplot(data = df,x="last_evaluation", kde = True,bins=20)
```

```
Out[45]: <Axes: xlabel='last_evaluation', ylabel='Count'>
```



```
In [46]: sns.countplot(x="number_project",hue="left",data=df)  
plt.xticks(rotation=90)
```

```
Out[46]: ([0, 1, 2, 3, 4, 5],  
          [Text(0, 0, '2'),  
           Text(1, 0, '3'),  
           Text(2, 0, '4'),  
           Text(3, 0, '5'),  
           Text(4, 0, '6'),  
           Text(5, 0, '7')])
```



```
In [47]: dfclus = df[["satisfaction_level", "last_evaluation", "left"]]
```

```
In [48]: dfclus
```

Out[48]:

	satisfaction_level	last_evaluation	left
0	0.38	0.53	1
1	0.80	0.86	1
2	0.11	0.88	1
3	0.72	0.87	1
4	0.37	0.52	1
...	...	...	...
14994	0.40	0.57	1
14995	0.37	0.48	1
14996	0.37	0.53	1
14997	0.11	0.96	1
14998	0.37	0.52	1

14999 rows × 3 columns

In [49]: `from sklearn.cluster import KMeans`

In [50]: `km=dfclus.iloc[:,:].values
kmeans = KMeans(n_clusters=3, random_state=0)
label = kmeans.fit_predict(dfclus)
labelarr = kmeans.fit_predict(km)`

In [51]: `label`

Out[51]: `array([1, 1, 1, ..., 1, 1, 1], dtype=int32)`

In [52]: `dfclus[label==0].describe()`

Out[52]:

	satisfaction_level	last_evaluation	left
count	4596.000000	4596.000000	4596.0
mean	0.453814	0.679569	0.0
std	0.152887	0.165613	0.0
min	0.120000	0.360000	0.0
25%	0.340000	0.550000	0.0
50%	0.510000	0.670000	0.0
75%	0.570000	0.810000	0.0
max	0.690000	1.000000	0.0

In [53]: `dfclus[label==1].describe()`

Out[53]:

	satisfaction_level	last_evaluation	left
count	3571.000000	3571.000000	3571.0
mean	0.440098	0.718113	1.0
std	0.263933	0.197673	0.0
min	0.090000	0.450000	1.0
25%	0.130000	0.520000	1.0
50%	0.410000	0.790000	1.0
75%	0.730000	0.900000	1.0
max	0.920000	1.000000	1.0

In [54]: `dfclus[label==2].describe()`

Out[54]:

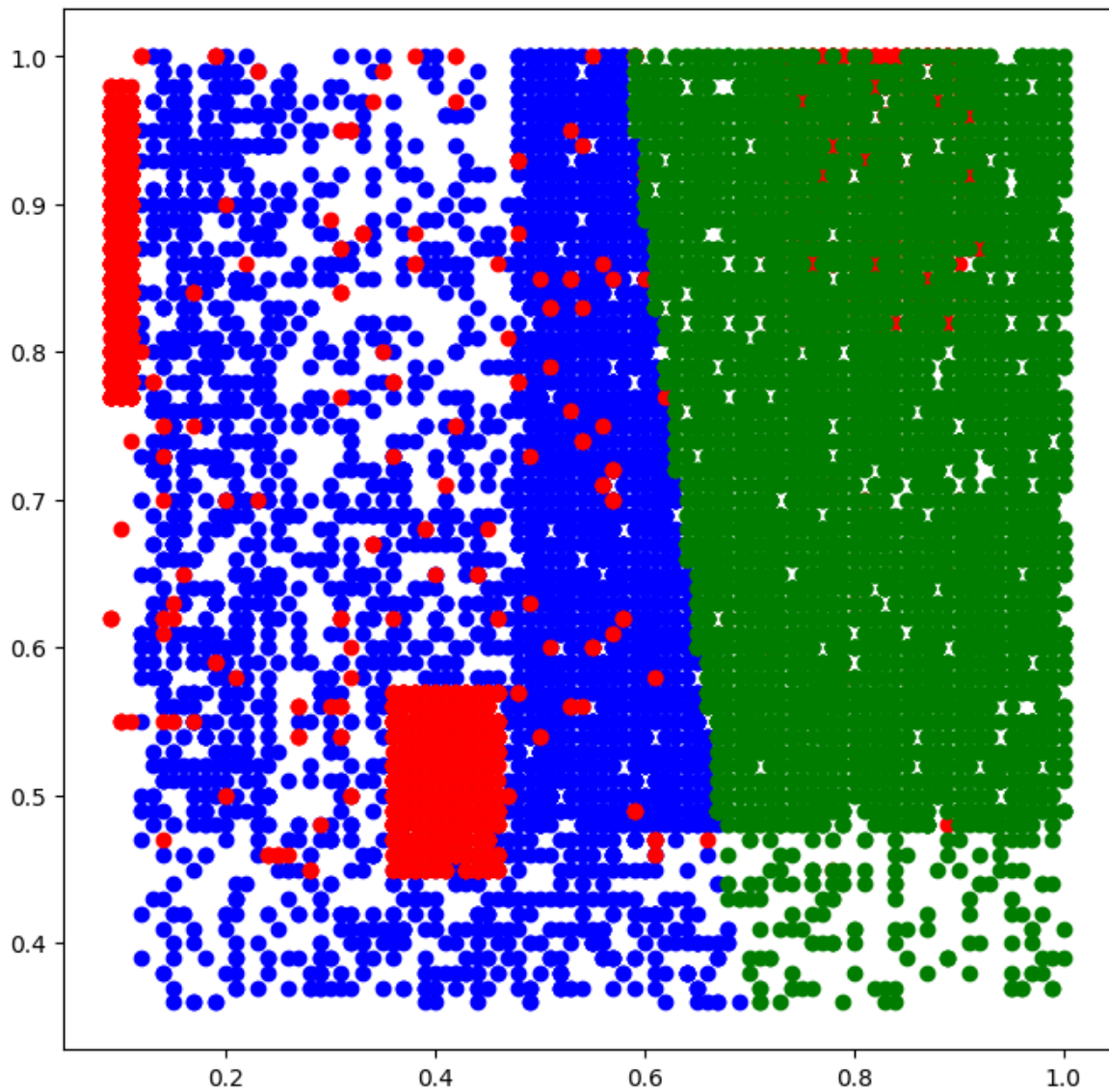
	satisfaction_level	last_evaluation	left
count	6832.000000	6832.000000	6832.0
mean	0.810095	0.739627	0.0
std	0.109845	0.154931	0.0
min	0.590000	0.360000	0.0
25%	0.720000	0.610000	0.0
50%	0.810000	0.740000	0.0
75%	0.910000	0.870000	0.0
max	1.000000	1.000000	0.0

In [55]: `km[label==0,1]`

Out[55]: `array([0.74, 0.69, 0.6 , ..., 0.94, 0.65, 0.73])`

In [56]: `plt.figure(figsize=(8,8))`
`plt.scatter(km[label==0,0],km[label==0,1],color="blue")`
`plt.scatter(km[label==1,0],km[label==1,1],color="red")`
`plt.scatter(km[label==2,0],km[label==2,1],color="green")`

Out[56]: `<matplotlib.collections.PathCollection at 0x1d936035190>`



```
In [57]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   satisfaction_level                    14999 non-null  float64
1   last_evaluation                      14999 non-null  float64
2   number_project                      14999 non-null  int64
3   average_monthly_hours               14999 non-null  int64
4   time_spend_company                  14999 non-null  int64
5   Work_accident                      14999 non-null  int64
6   left                               14999 non-null  int64
7   promotion_last_5years               14999 non-null  int64
8   sales                              14999 non-null  object
9   salary                              14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB
```

```
In [58]: df_numerical=df.select_dtypes(include=['int64','float64'])
df_categorical=df.select_dtypes(include=['object'])
```

```
In [59]: df_numerical=df.select_dtypes(include=['int64','float64'])
```

```
In [60]: df_categorical=df.select_dtypes(include=['object'])
```

```
In [61]: df_converted = pd.get_dummies(data=df_categorical)
```

```
In [62]: df_converted.head()
```

```
Out[62]:
```

	sales_IT	sales_RandD	sales_accounting	sales_hr	sales_management	sales_marketing
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	False	False	False	False	False
4	False	False	False	False	False	False



```
In [63]: dfn = pd.concat([df_numerical, df_converted], axis=1, join="inner")
```

```
In [64]: dfn.shape
```

```
Out[64]: (14999, 21)
```

```
In [65]: dfn.head()
```

```
Out[65]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	time_spend
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

5 rows × 21 columns



```
In [66]: x = dfn.drop("left",axis=1)
y = dfn["left"]
```

```
In [67]: from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest = train_test_split(x,y,test_size=0.2,random_state=123)
```

```
In [68]: xtrain.shape,ytrain.shape,xtest.shape,ytest.shape
```

```
Out[68]: ((11999, 20), (11999,), (3000, 20), (3000,))
```

```
In [69]: ((11999, 20), (11999,), (3000, 20), (3000,))
```

```
Out[69]: ((11999, 20), (11999,), (3000, 20), (3000,))
```

```
In [70]: ytrain.value_counts()
```

```
Out[70]: left
0      9137
1      2862
Name: count, dtype: int64
```

```
In [72]: pip install imbalanced-learn
```

Collecting imbalanced-learn

Downloading imbalanced_learn-0.14.1-py3-none-any.whl.metadata (8.9 kB)

Requirement already satisfied: numpy<3,>=1.25.2 in c:\users\chann\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (2.1.1)

Requirement already satisfied: scipy<2,>=1.11.4 in c:\users\chann\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.14.1)

Requirement already satisfied: scikit-learn<2,>=1.4.2 in c:\users\chann\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.5.2)

Collecting sklearn-compat<0.2,>=0.1.5 (from imbalanced-learn)

Downloading sklearn_compat-0.1.5-py3-none-any.whl.metadata (20 kB)

Requirement already satisfied: joblib<2,>=1.2.0 in c:\users\chann\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.4.2)

Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\chann\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (3.5.0)

Downloading imbalanced_learn-0.14.1-py3-none-any.whl (235 kB)

Downloading sklearn_compat-0.1.5-py3-none-any.whl (20 kB)

Installing collected packages: sklearn-compat, imbalanced-learn

```
----- 0/2 [sklearn-compat]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 1/2 [imbalanced-learn]
----- 2/2 [imbalanced-learn]
```

Successfully installed imbalanced-learn-0.14.1 sklearn-compat-0.1.5

Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.2 -> 26.1

[notice] To update, run: C:\Users\Chann\AppData\Local\Programs\Python\Python312\python.exe -m pip install --upgrade pip

```
In [73]: from imblearn.over_sampling import SMOTE
```

```
In [74]: sm = SMOTE(random_state = 2)
xtrainres, ytrainres = sm.fit_resample(xtrain, ytrain)
```

```
In [75]: ytrainres.value_counts()
```

```
Out[75]: left
0      9137
1      9137
Name: count, dtype: int64
```

```
In [76]: from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
import sklearn.metrics as metrics
```

```
In [77]: logreg = LogisticRegression(solver='lbfgs', max_iter=10000)
```

```
In [78]: print(cross_val_score(logreg, xtrainres, ytrainres, cv=5).mean())

0.8055723502980449
```

```
In [79]: logreg.fit(xtrainres,ytrainres)
ypred = logreg.predict(xtest)
```

```
In [80]: from sklearn.metrics import classification_report
```

```
In [81]: metrics.confusion_matrix(ytest,ypred)
```

```
Out[81]: array([[1832,  459],
               [ 229,  480]])
```

```
In [82]: print(classification_report(ytest,ypred))
```

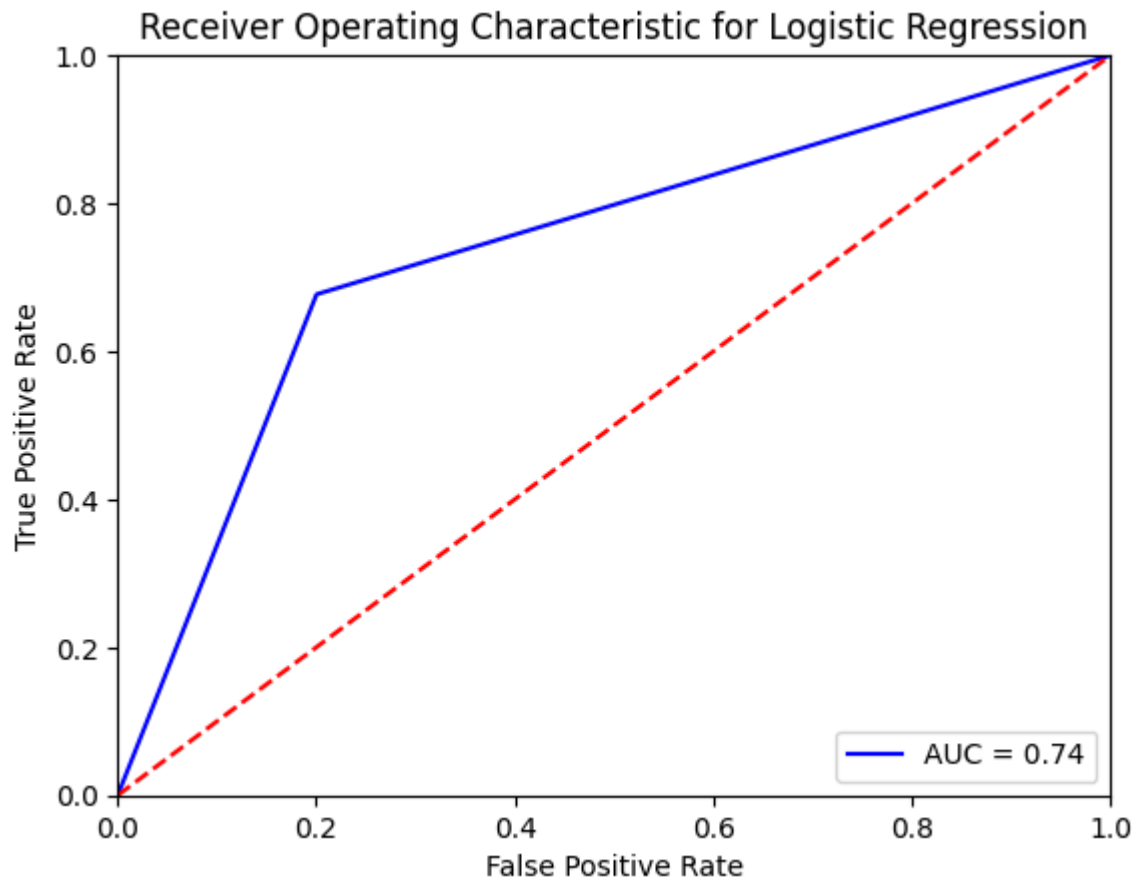
	precision	recall	f1-score	support
0	0.89	0.80	0.84	2291
1	0.51	0.68	0.58	709
accuracy			0.77	3000
macro avg	0.70	0.74	0.71	3000
weighted avg	0.80	0.77	0.78	3000

```
In [83]: roc_auc_score(ytest,ypred)
```

```
Out[83]: np.float64(0.7383303402841437)
```

```
In [84]: fpr, tpr, threshold = metrics.roc_curve(ytest, ypred)
print(fpr)
print(tpr)
print(threshold)
roc_auc = metrics.auc(fpr, tpr)
print(roc_auc)
# method 1:plt
plt.title('Receiver Operating Characteristic for Logistic Regression')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0,1], [0,1], 'r--')
plt.xlim([0,1])
plt.ylim([0,1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
```

```
[0.          0.20034919 1.          ]
[0.          0.67700987 1.          ]
[inf  1.  0.]
0.7383303402841437
```



```
In [85]: randm=RandomForestClassifier(max_depth=5)
```

```
In [86]: print(cross_val_score(randm, xtrainres, ytrainres, cv=5).mean())
```

```
0.9486155606321652
```

```
In [87]: randm.fit(xtrainres,ytrainres)
ypred1=randm.predict(xtest)
```

```
In [88]: metrics.confusion_matrix(ytest,ypred1)
```

```
Out[88]: array([[2218,  73],
               [ 52, 657]])
```

```
In [89]: print(classification_report(ytest,ypred1))
```

	precision	recall	f1-score	support
0	0.98	0.97	0.97	2291
1	0.90	0.93	0.91	709
accuracy			0.96	3000
macro avg	0.94	0.95	0.94	3000
weighted avg	0.96	0.96	0.96	3000

```
In [90]: roc_auc_score(ytest,ypred1)
```

```
Out[90]: np.float64(0.947396724411892)
```

```
In [91]: fpr, tpr, threshold = metrics.roc_curve(ytest, ypred1)
print(fpr)
```

```

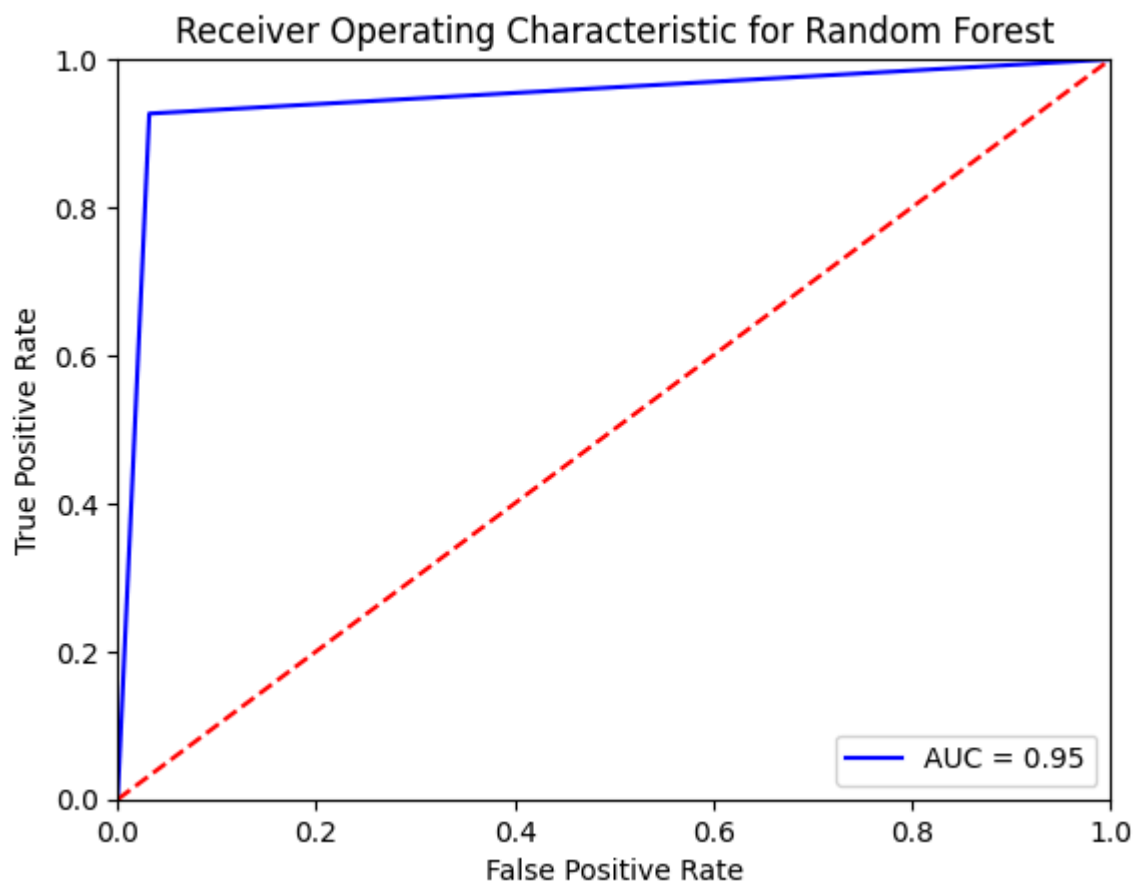
print(tpr)
print(threshold)
roc_auc = metrics.auc(fpr, tpr)
print(roc_auc)
# Method 1: plt
plt.title('Receiver Operating Characteristic for Random Forest')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0,1], 'r--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()

```

```

[0.      0.03186381 1.      ]
[0.      0.92665726 1.      ]
[inf  1.  0.]
0.947396724411892

```



```
In [92]: from sklearn.ensemble import GradientBoostingClassifier
```

```
In [103... gb = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=1.0,
    max_depth=1,
    random_state=42
)
```

```
In [104... print(cross_val_score(gb, xtrainres, ytrainres, cv=5).mean())
```

```
0.9476854628512725
```

```
In [105... gb.fit(xtrainres, ytrainres)
```

```
Out[105... GradientBoostingClassifier
GradientBoostingClassifier(learning_rate=1.0, max_depth=1, random_state=42)
```

```
In [106... ypred2 = gb.predict(xtest)
```

```
In [107... metrics.confusion_matrix(ytest,ypred2)
```

```
Out[107... array([[2168, 123],
        [ 46, 663]])
```

```
In [108... print(classification_report(ytest,ypred2))
```

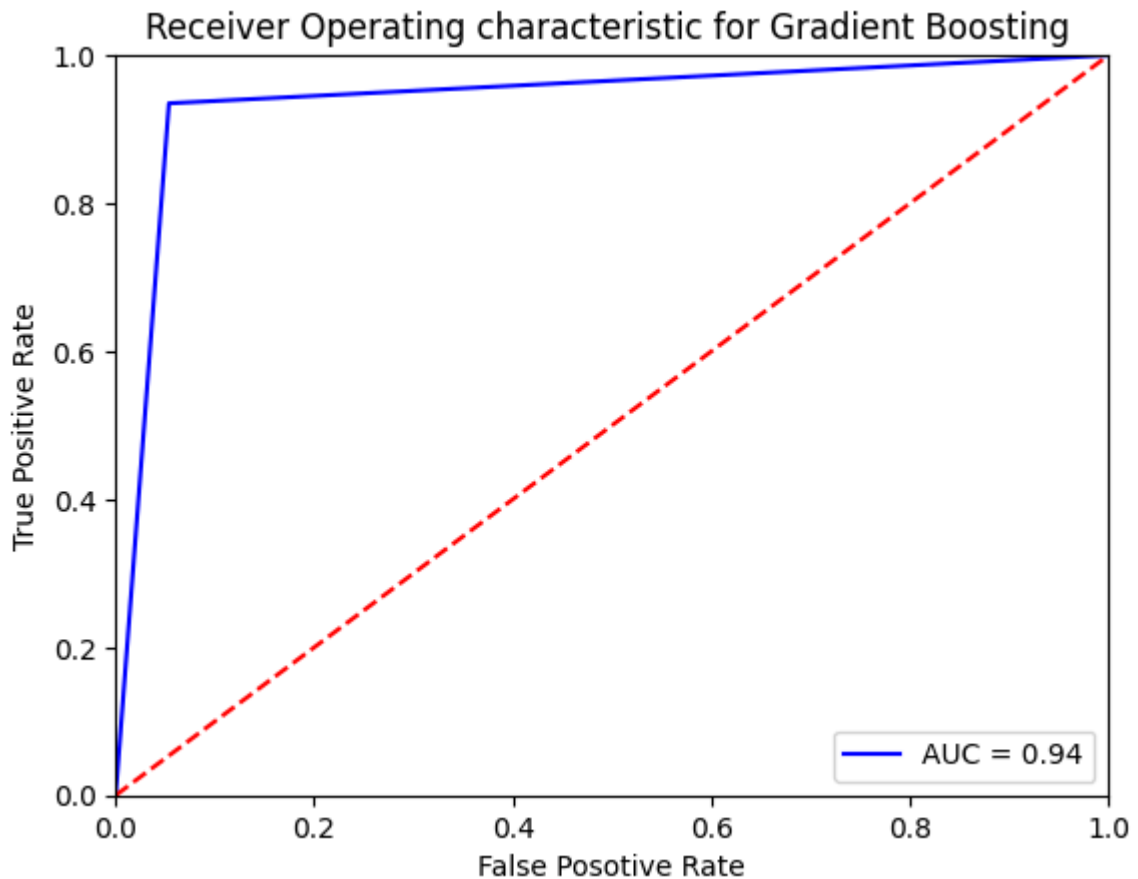
	precision	recall	f1-score	support
0	0.98	0.95	0.96	2291
1	0.84	0.94	0.89	709
accuracy			0.94	3000
macro avg	0.91	0.94	0.92	3000
weighted avg	0.95	0.94	0.94	3000

```
In [109... roc_auc_score(ytest,ypred2)
```

```
Out[109... np.float64(0.9407157707322269)
```

```
In [110... fpr, tpr, threshold = metrics.roc_curve(ytest, ypred2)
print(fpr)
print(tpr)
print(threshold)
roc_auc = metrics.auc(fpr, tpr)
print(roc_auc)
# method 1: plt
plt.title('Receiver Operating characteristic for Gradient Boosting ')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0,1],[0,1], 'r--')
plt.xlim([0,1])
plt.ylim([0,1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Posotive Rate')
plt.show()
```

```
[0.          0.05368835 1.          ]
[0.          0.93511989 1.          ]
[inf  1.  0.]
0.9407157707322269
```

```
In [111...] col = xtrainres.columns
```

```
In [112...] col
```

```
Out[112...] Index(['satisfaction_level', 'last_evaluation', 'number_project',
      'average_monthly_hours', 'time_spend_company', 'Work_accident',
      'promotion_last_5years', 'sales_IT', 'sales_RandD', 'sales_accounting',
      'sales_hr', 'sales_management', 'sales_marketing', 'sales_product_mng',
      'sales_sales', 'sales_support', 'sales_technical', 'salary_high',
      'salary_low', 'salary_medium'],
      dtype='object')
```

```
In [113...] feature_labels = np.array(col)
```

```
In [114...] importance = randm.feature_importances_
feature_indexes_by_importance = importance.argsort()
for index in feature_indexes_by_importance:
    print('{}-{:0.2f}%'.format(feature_labels[index], (importance[index] * 100.0)))
```

```

sales_product_mng-0.01%
sales_marketing-0.01%
sales_management-0.02%
sales_RandD-0.03%
sales_IT-0.03%
sales_accounting-0.05%
sales_support-0.08%
sales_hr-0.10%
sales_sales-0.17%
sales_technical-0.20%
promotion_last_5years-0.24%
salary_medium-0.38%
salary_high-0.81%
salary_low-1.52%
Work_accident-2.99%
last_evaluation-11.35%
average_monthly_hours-11.53%
number_project-16.51%
time_spend_company-22.64%
satisfaction_level-31.33%

```

```
In [115... predict_probability = randm.predict_proba(xtest)
```

```
In [116... predict_probability[:,1]
```

```
Out[116... array([0.05681267, 0.12539914, 0.10025421, ..., 0.73230309, 0.06546678,
        0.13775681])
```

```
In [120... zone = []
prob = []

for i in predict_probability[:, 1]:
    prob.append(i)

    if i <= 0.2:
        zone.append("Safe Zone")
    elif i > 0.2 and i <= 0.6:
        zone.append("Low Risk Zone")
    elif i > 0.6 and i <= 0.9:
        zone.append("Medium Risk Zone")
    else:
        zone.append("High Risk Zone")
```

```
In [121... categories = ["Safe Zone","Low Risk Zone","Medium Risk Zone ","High Risk Zone "]
color = ["Green","Yellow","Orange","Red"]
```

```
In [122... colordict = dict(zip(categories, color))
```

```
In [123... clr = pd.DataFrame({"zone":zone,"probability":prob})
```

```
In [124... clr["zone"].unique()
```

```
Out[124... array(['Safe Zone', 'High Risk Zone', 'Medium Risk Zone', 'Low Risk Zone'],
      dtype=object)
```

```
In [128... clr["Color"] = clr["zone"].apply(lambda x: colordict)
```

```
In [130... print(colordict)
```

```
{'Safe Zone': 'Green', 'Low Risk Zone': 'Yellow', 'Medium Risk Zone ': 'Orange',  
'High Risk Zone ': 'Red'}
```

```
In [131... colordict = {  
    "Safe Zone": "green",  
    "Low Risk Zone": "yellow",  
    "Medium Risk Zone": "orange",  
    "High Risk Zone": "red"  
}
```

```
In [132... clr["Color"] = clr["zone"].apply(lambda x: colordict.get(x, "black"))
```

```
In [133... clr.head(10)
```

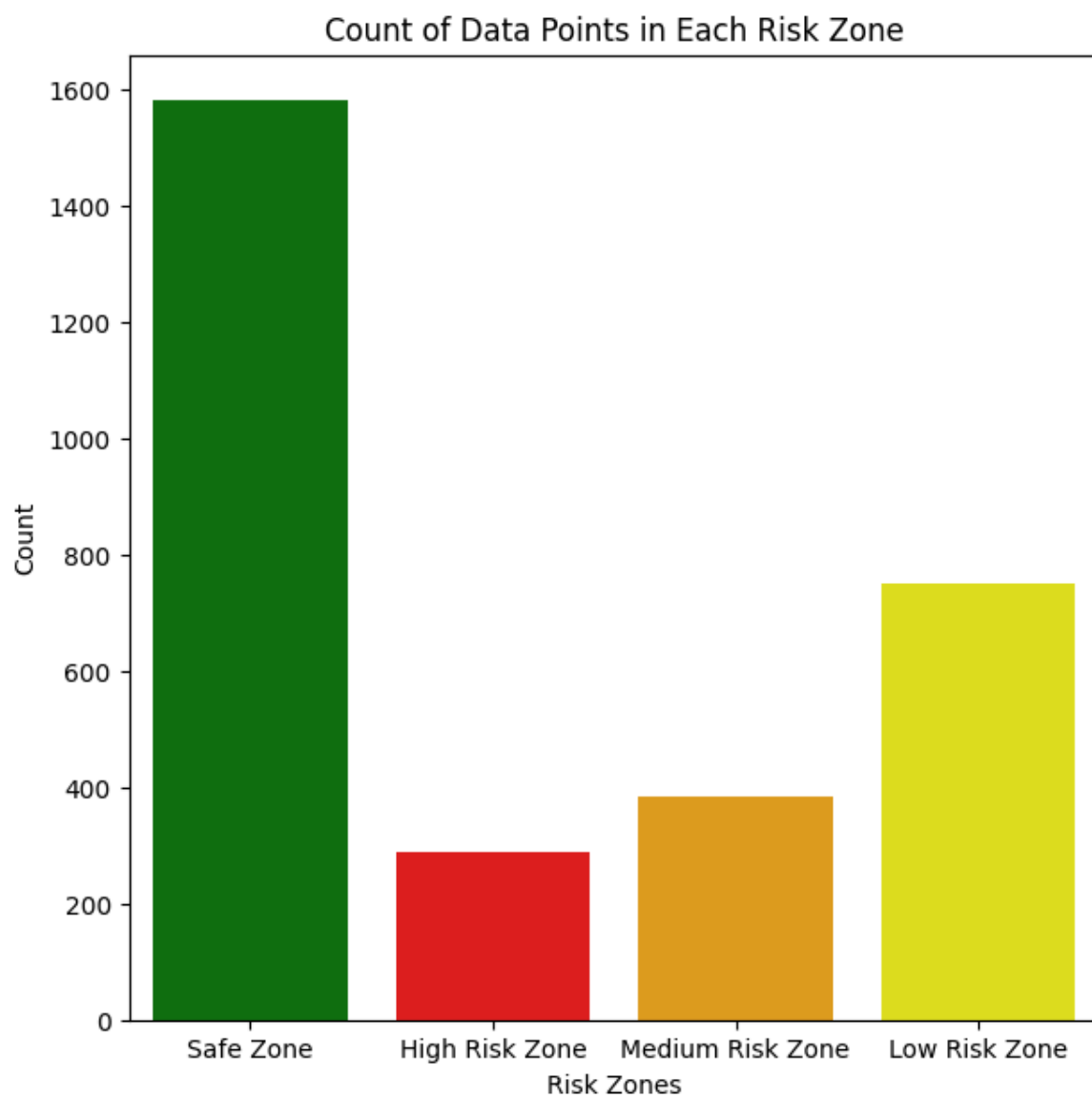
```
Out[133...
```

	zone	probability	Color
0	Safe Zone	0.056813	green
1	Safe Zone	0.125399	green
2	Safe Zone	0.100254	green
3	Safe Zone	0.093217	green
4	Safe Zone	0.135327	green
5	Safe Zone	0.080853	green
6	High Risk Zone	0.951724	red
7	Medium Risk Zone	0.773779	orange
8	Safe Zone	0.099816	green
9	Safe Zone	0.059963	green

```
In [134... color= clr["Color"].tolist()
```

```
In [135... c = ["Green", "Red", "Orange", "Yellow"]
```

```
In [136... data_df = pd.DataFrame({'Risk Zones': zone})  
plt.figure(figsize=(7, 7))  
sns.countplot(data=data_df, x='Risk Zones', palette=c) # Specify 'x' as the cate  
plt.xlabel('Risk Zones')  
plt.ylabel('Count')  
plt.title('Count of Data Points in Each Risk Zone')  
plt.show()
```



In []: